C++11 STL 容器 API 手册

每个容器：API 表格 → 独立使用示例

以 C++11 为基准，API 表与独立程序均采用该标准可用的接口。各容器先列常用 API、含义与复杂度，随后给出该容器独立可运行的示例、预期输出和使用要点。

左侧书签分类

pair · vector · array · string · deque · list · forward_list
map · multimap · set · multiset
unordered_map · unordered_multimap
unordered_set · unordered_multiset
stack · queue · priority_queue

如何读表

c 表示当前容器，other 表示同类型容器，pos / first / last 为合法迭代器，key 为键，value 为元素；n 为当前元素数量，k 为处理或匹配的元素数量。

O(1) 表示常数级操作；“摊还”考虑整个操作序列的成本；“平均”表示依赖哈希分布等条件。复杂度通常按元素操作或比较次数计算，未把字符串键长度、用户构造函数成本当作常数之外展开。

> 这是常用 API 手册，不穷举分配器重载、执行策略与所有标准版本接口。除适配器外，多数容器都有 begin/end、empty/size 等接口，但 forward_list 没有 size，array 没有 clear。

生成日期：2026-09-30

C++11 / 版本边界与替代写法

这份手册采用 C++11 的语言和标准库接口。下表只说明版本差异；右列给出本版使用的写法。

新版写法（C++11 不支持）	C++11 写法 / 注意事项
`contains(key) [C++20]`	find(key) != end()，或 count(key) != 0
`try_emplace [C++17]`	按需求用 emplace/insert；要避免重复键时构造 value，先 find 再构造并插入
`insert_or_assign [C++17]`	可默认构造的值类型可用 m[key] = value；其他情况先 find，再赋值或插入
`extract / 关联容器 merge [C++17]`	保存值后 erase + insert；list/forward_list 原有的 merge 在 C++11 可用
`auto [key,value] [C++17]`	通过 pair.first、pair.second 或 it->first、it->second 访问
`std::erase / std::erase_if [C++20]`	序列用 erase-remove；关联容器循环中使用 it = c.erase(it)
`string::starts_with/ends_with [C++20]`	先检查长度，再用 compare 比较首尾子串
`generic lambda: [](auto x) [C++14]`	为参数明确指定类型，如 [](int x)

C++11 本身可以使用的语法

auto 类型推导、范围 for、花括号初始化、nullptr、普通 lambda、移动语义、emplace、unordered 容器均可使用。为了便于辨认容器类型，本手册主要显式书写类型。

> 返回值也有版本差异：C++11 的 vector/deque/list::emplace_back、deque/list/forward_list::emplace_front，以及 stack/queue::emplace 返回 void；string::data 返回 const char*。

有序与 unordered 容器：对应关系

unordered 不是所有容器都能加的前缀。它对应四类基于键查询的容器：set、multiset、map、multimap。vector、list、deque 没有标准 unordered 版本。

有序关联容器	无序关联容器	保存内容
`set<Key>`	unordered_set<Key>	唯一键；用于集合与去重
`multiset<Key>`	unordered_multiset<Key>	允许重复键的集合
`map<Key,T>`	unordered_map<Key,T>	唯一键，每个键对应一个值
`multimap<Key,T>`	unordered_multimap<Key,T>	允许重复键，每个元素仍是一对键值

共同点与主要区别

项目	有序版本	unordered 版本
查找依据	按 key	按 key
组织方式	常见为平衡搜索树	哈希表与桶
遍历顺序	按比较器排序；默认通常升序	没有全局排序保证
判断键等价	!comp(a,b) && !comp(b,a)	key_equal(a,b)，等价键须有相同 hash
`find / 单元素普通插入`	O(log n)	平均 O(1)，最坏 O(n)
`lower_bound / upper_bound`	有	没有
`equal_range`	有	有；不是大小范围查询
迭代器	双向	前向
`reserve / rehash / 桶接口`	没有	有
插入后的旧迭代器	保持有效	若发生 rehash 则失效

> n 是当前元素数；k 是匹配元素数。unordered 的“平均常数”不是最坏保证，读取所有 k 个匹配项仍需至少 O(k) 次访问。有序容器也不保证按插入先后遍历。

对比 / set 与 unordered_set

两个容器都只保存 key，不保存独立的 value；等价 key 只保留一份。

项目	set	unordered_set
插入 3、1、2、3	三个元素，遍历为 1、2、3	三个元素，遍历顺序不保证
insert(value) 返回值	pair<iterator,bool>	pair<iterator,bool>
find / insert / erase(key)	O(log n)	平均 O(1)，最坏 O(n)
count(key)	0 或 1；O(log n)	0 或 1；平均 O(1)，最坏 O(n)
第一个不小于 x 的值	成员 lower_bound(x)	没有直接对应 API
最小元素	非空时 *begin()	begin() 不代表最小值
operator[] / at	都没有	都没有
反向迭代	rbegin/rend	没有
元素能否通过迭代器修改	不能修改 key	不能修改 key

什么时候选哪个？

判断整数是否出现过、去重、查找补数：通常使用 unordered_set。需要按值从小到大输出、找前驱后继或做有序范围查询：使用 set。

插入 n 次的总成本

普通 set 插入 n 个不同元素，总成本 O(n log n)。unordered_set 的平均总成本 O(n)，严重碰撞时最坏可达 O(n^2)。只说“循环一次”不足以判断整体复杂度，还必须计算每次循环内的容器操作。

> std::find(s.begin(),s.end(),x) 会线性扫描。已经选用 set 或 unordered_set 时，应优先用成员 s.find(x)，才能利用容器自身的查找结构。

对比 / multiset 与 unordered_multiset

两个容器都允许等价 key 重复出现。每个重复值也是一个独立元素。

项目	multiset	unordered_multiset
插入 2、1、2	遍历为 1、2、2	保留两个 2，无全局排序保证
`insert(value)` 返回值	iterator	iterator
`find` / 单元素插入	O(log n)	平均 O(1)，最坏 O(n)
`count(key)`	O(log n+k)	平均 O(1+k)，最坏 O(n)
`equal_range(key)`	取得有序等价区间，O(log n)	取得等价组，平均 O(1+k)，最坏 O(n)
`erase(key)`	删除全部同键元素，O(log n+k)	删除全部同键元素，平均 O(1+k)，最坏 O(n)
仅删除一次	find 后 erase(it)	find 后 erase(it)
大小边界查询	lower_bound / upper_bound	没有
重复键的判定	比较器等价	key_equal 等价

选择建议

需要维护可重复的有序值、找最接近的值或取当前最小值时，用 multiset。只需要保存重复项并按键查询时，可考虑 unordered_multiset。

如果只需要知道每个值出现几次，而不需要逐份保存，unordered_map<Key,int> 计数往往更直接。例如一万个相同整数可以表现为一个键和一个计数。

> 无论哪一种，erase(2) 都不是“只删一个 2”。先 find(2)，确认没有返回 end，再 erase(it)，才是删除一个匹配元素。

对比 / map 与 unordered_map

两个容器的 key 唯一，每个元素为 pair<const Key,T>：key 不能通过普通迭代器修改，value 可以通过可写迭代器修改。

项目	map	unordered_map
遍历顺序	按 key 比较器排序	无全局排序保证
`find / at / 普通 insert`	O(log n)	平均 O(1)，最坏 O(n)
`operator[](key)`	缺失时插入默认构造的 value	相同语义
`at(key)`	缺失时抛 out_of_range	相同语义
`insert / emplace`	已有 key 不覆盖	相同语义
`insert` 的返回值	pair<iterator,bool>	pair<iterator,bool>
`lower_bound / upper_bound`	有	没有
`reserve / rehash`	没有	有
插入时位置稳定性	旧元素迭代器保持有效	rehash 会使旧迭代器失效

刷题中的两个常见用法

统计单词频率、两数之和的值到下标映射：通常选 unordered_map。按键输出统计结果、按时间戳查第一个不早于某时刻的记录：选 map。

一个 key 也能对应一组 value

map<string,vector<string>> 或 unordered_map<string,vector<string>> 都能把一组字符串放在同一个 key 下。此时“键唯一”不代表业务上只能保存一个字符串，而是每个 key 只对应一个 vector 对象。

> 只检查 key 是否存在，使用 find(key) != end()，不要写 m[key]。后者可能修改容器并插入一条原本没有的记录。

对比 / multimap 与 unordered_multimap

两个容器都可以保存多个相同 key 的键值对；每次插入是一个独立元素，而不是向同一个 vector 追加。

项目	multimap	unordered_multimap
全局遍历	按 key 排序	没有按 key 排序的保证
`insert / emplace` 返回值	iterator	iterator
`find(key)`	返回一个匹配元素或 end	返回一个匹配元素或 end
`equal_range(key)`	全部等价键区间；O(log n)	全部等价键组；平均 O(1+k)，最坏 O(n)
读取全部 `k` 个 `value`	另需 O(k) 次遍历	另需 O(k) 次遍历
`count / erase(key)`	O(log n+k)	平均 O(1+k)，最坏 O(n)
`operator[] / at`	没有	没有
按 `key` 大小范围查询	有 lower_bound / upper_bound	没有
只删除一个键值对	使用合法迭代器 erase(it)	相同语义

与你的字母异位词分组题的关系

排序后的字符串可作为 key，原字符串作为 value。multimap 可以用 equal_range 收集同一组；unordered_multimap 也有这个接口，但不提供按 key 大小排列的结果。

如果最终结果本来就是一组组 vector，更直接的结构常是 unordered_map<string,vector<string>>。这样每个 key 只存一次，值直接是该组字符串，不必再通过 equal_range 收集。

> 改为 unordered 容器，不会消除生成 key 的成本。例如 n 个最长为 L 的字符串，各自排序仍需要 O(n·L log L) 次比较量级；哈希字符串也需要读取字符，不能把所有工作都算成 O(n)。

序列式与关联式容器：核心区别

序列式容器按“元素所在的位置”组织数据；关联式容器按“键”组织和查找数据。“序列式”不是“自动排序”，“关联式”也不等于“只能是键值对”。

维度	序列式容器	关联式容器
核心概念	第几个、前后位置、插入位置	key、键等价、按键查询
元素顺序	由插入位置和后续操作决定	有序类按比较器；无序类无全局排序
重复元素	通常允许	取决于是否为 multi 容器
随机下标访问	vector/array/deque 支持，链表不支持	不提供按第几个元素的下标
按值查找	没有额外索引时通常 O(n)	有序 O(log n)；哈希平均 O(1)
插入接口	push_back、insert 等，因容器而异	insert、emplace；不指定普通序列下标
典型用途	数组、缓冲区、顺序遍历、位置操作	集合、去重、计数、键到值的索引

同样写 c[3]，含义可能不同

vector<int> 的 v[3] 指第 4 个元素，必须已有这个位置。map<int,int> 的 m[3] 指 key 等于 3 的值；key 不存在会插入。它不是“第 4 条记录”。set 则完全没有 []。

有序和无序都属于按键组织的数据结构

教学中常把两者统称为关联式容器。C++ 标准在规范上将“关联容器”和“无序关联容器”分别列出；本章采用广义分类，同时明确两者区别。

> vector 排序后仍是序列式容器：排序不会改变容器类型，也不会让后续 push_back 自动维持排序。有序关联容器则会在插入时维护按 key 的排列关系。

容器分类 / 全部类型与选型速查

下面覆盖本手册已介绍的全部容器及相关工具类型。

类型	分类	顺序与典型用途
vector	序列式	连续动态数组，按位置访问
array	序列式	固定长度连续数组
deque	序列式	双端增删，随机访问，非整体连续
list	序列式	双向链表，已知节点位置操作
forward_list	序列式	单向链表，按前驱操作
set / multiset	有序关联式	按键排序；唯一键 / 重复键
map / multimap	有序关联式	有序键值表；唯一键 / 重复键
unordered_set / unordered_multiset	无序关联式	哈希集合；唯一键 / 重复键
unordered_map / unordered_multimap	无序关联式	哈希键值表；唯一键 / 重复键
stack	容器适配器	后进先出，默认基于 deque
queue	容器适配器	先进先出，默认基于 deque
priority_queue	容器适配器	按优先级取堆顶，默认基于 vector
string	字符串类	具有序列容器式接口；标准单列字符串库
pair	工具类型，不是容器	保存两个成员；没有容器式遍历接口

快速选择顺序

需要按位置存取：先考虑 vector。需要只判断存在：unordered_set。需要按键计数或映射：unordered_map。需要有序范围查找：set/map。需要 FIFO/LIFO：queue/stack。只关心当前最值：priority_queue。

> priority_queue 不是有序关联容器。它只保证堆顶符合优先级，并没有提供按任意 key 查找、遍历全部元素或 lower_bound 的接口。

迭代器：位置、类型与 const

迭代器是表示容器中某个位置的对象，可以先类比 C 的指针理解，但它不一定是真正的指针，也不一定支持指针的全部运算。

类型与变量分开看

写法	含义
`std::vector<int>`	容器类型
`v`	容器变量
`std::vector<int>::iterator`	可通过它修改元素的迭代器类型
`std::vector<int>::iterator it = v.begin();`	定义变量 it，使其指向首元素
`*it`	取得当前位置的元素
`++it`	移动到下一个元素
`it != v.end()`	判断是否还没到尾后位置
`it->first / it->second`	map/multimap 中当前元素的 key / value

begin 和 end 的位置

```
元素:        10       20       30      [尾后位置]
位置:      begin()                    end()
区间:      [begin(), end())
```

end() 不指向最后一个元素，而是最后一个元素之后的位置。它可用于比较，不能解引用。空容器的 begin() == end()；任何元素访问之前都需要保证位置有效。

iterator 与 const_iterator

类型	能否 ++it	能否修改元素
`iterator`	可以	通常可以；set 的键、map 的 key 除外
`const_iterator`	可以	不可以通过该迭代器修改
`const iterator`	不可以	可能可以；const 修饰迭代器变量本身

> const_iterator 限制的是通过它写元素，不是限制它移动，也不保证其他代码不能修改容器。iterator 通常能转换成 const_iterator，反过来不行。const 容器的 begin() 返回 const_iterator；非 const 容器的 cbegin() 也返回 const_iterator。

迭代器 / 操作 API 与能力分类

迭代器工具函数主要来自 <iterator>。操作是否合法，取决于迭代器类别与实际位置。

操作 / API	作用	条件与成本
`begin/end; cbegin/cend`	首元素 / 尾后位置；只读版本	一般 O(1)
`rbegin/rend; crbegin/crend`	反向遍历边界	容器须提供反向迭代器
`*it / it->member`	读写元素 / 访问元素成员	必须可解引用，不能是 end
`++it / it++`	前进；后者返回移动前的副本	不能从 end 再前进
`--it`	后退	双向及以上；不能从 begin 后退
`it + n / it - n / it[n]`	按偏移访问	随机访问；结果须在有效范围
`last - first`	位置差	同一序列的随机访问迭代器
`std::next(it,n)`	返回前进后的副本，不改变 it	非随机访问通常 O(n)
`std::prev(it,n)`	返回后退后的副本，不改变 it	至少双向；须能合法后退
`std::advance(it,n)`	直接移动 it 本身	随机访问 O(1)，其他通常线性
`std::distance(first,last)`	计算从 first 到 last 的距离	随机访问 O(1)，其他线性
`reverse_iterator::base()`	对应的正向迭代器	指向反向当前元素的后一个位置

容器支持哪些能力?

类别	常见例子	能力
前向	forward_list、unordered_*	++，不能 -- 或 +n
双向	list、map、multimap、set	++ 和 --，不能 +n
随机访问	vector、array、deque、string	++、--、+n、相减
输入 / 输出	输入流迭代器 / back_inserter	读入 / 写出；不要假设可重复遍历

> 不同容器的迭代器不能混在一起比较或组成算法区间。std::sort 要求随机访问迭代器，因此 list 应使用自己的 sort()。priority_queue、queue、stack 不提供 begin/end。

迭代器 / 修改容器后的失效规则

失效意味着旧位置不再能按原来的方式使用。指针数值看起来没变，也不代表它仍然有效。

容器 / 修改	失效影响
`vector` 发生重新分配	所有旧迭代器、元素指针和引用、旧 end 失效
`vector` 插入但不重新分配	插入点及其之后的迭代器和引用失效，含旧 end
`vector erase`	删除点及之后的位置失效，含旧 end
`list / forward_list` 插入	原有元素的迭代器和引用仍有效
`list / forward_list` 删除	仅被删除元素的迭代器和引用失效
`map / set` 等有序容器插入	原有元素迭代器和引用仍有效
`map / set` 等有序容器删除	仅被删元素失效
`unordered_* 发生 rehash`	所有迭代器失效；已有元素的指针和引用仍有效
`unordered_* erase`	仅被删元素的迭代器和引用失效
`deque` 两端插入	迭代器失效；已有元素引用仍有效
`deque` 中间插入	所有迭代器和元素引用失效
`array`	固定存储，无扩容；注意对象本身生命周期

删除循环的正确思路

删除时让 it 接收 erase(it) 返回的后继；不删除时才 ++it。不能先 erase(it)，然后再对已经失效的 it 执行 ++。

vector 逐个删除会反复移动后续元素，最坏可达 $O(n^2)$。批量删除常使用 erase-remove；这里下一页选择 map 演示安全删除。

const_iterator 不会防止失效

它只能禁止通过该迭代器修改元素。如果其他操作使 vector 扩容，之前保存的 const_iterator 同样失效。不要把“只读”理解为“位置永远稳定”。

> 本表是常用规则摘要，不覆盖 swap、移动赋值、deque 所有删除位置和 string 全部操作。跨修改保存位置之前，应检查具体接口的失效规则。

迭代器 / vector 读写与只读遍历

完整独立程序：iterator_vector.cpp。C++11，直接复制即可编译，不依赖前文定义。

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{10, 20, 30};
    // iterator 可以修改元素
    for (std::vector<int>::iterator it = v.begin();
         it != v.end(); ++it) {
        *it += 1;
    }
    // const_iterator 只能读取，但仍可执行 ++it
    for (std::vector<int>::const_iterator it = v.cbegin();
         it != v.cend(); ++it) {
        std::cout << *it << ' ';
    }
    std::cout << '\n';
    // const 容器的 begin 也产生只读迭代器
    const std::vector<int> frozen{7, 8};
    std::vector<int>::const_iterator p = frozen.begin();
    if (p != frozen.end()) std::cout << *p << '\n';
    return 0;
}
```

预期输出

```
11 21 31
7
```

把只读遍历中的 *it 改成赋值表达式会编译失败。两段 for 中的 it 各自只在自己的循环内有效。

迭代器 / map 遍历并安全删除

完整独立程序：iterator_map_erase.cpp。C++11，直接复制即可编译，不依赖前文定义。

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> counts;
    counts["apple"] = 2;
    counts["banana"] = 0;
    counts["pear"] = 0;
    std::map<std::string, int>::iterator it = counts.begin();
    while (it != counts.end()) {
        if (it->second == 0) {
            it = counts.erase(it); // 删除后接收新位置
        } else {
            ++it;                  // 没有删除才前进
        }
    }
    for (std::map<std::string, int>::const_iterator p
             = counts.cbegin(); p != counts.cend(); ++p) {
        std::cout << p->first << ':' << p->second << '\n';
    }
    return 0;
}
```

预期输出

```
apple:2
```

map 的 it->first 是只读 key，it->second 是 value。只读的输出循环使用 const_iterator；删除循环使用 iterator，清楚表达操作意图。

迭代器 / next、advance 与反向遍历

完整独立程序：iterator_navigation.cpp。C++11，直接复制即可编译，不依赖前文定义。

```
#include <iostream>
#include <iterator>
#include <list>
#include <vector>

int main() {
    std::list<int> nodes{10, 20, 30};
    std::list<int>::iterator it = nodes.begin();
    std::list<int>::iterator next = std::next(it, 2);
    std::cout << "original=" << *it
              << " next=" << *next << '\n';
    std::advance(it, 1); // 修改 it 本身，使其指向 20
    std::cout << "advanced=" << *it << '\n';
    std::cout << "distance="
              << std::distance(nodes.begin(), nodes.end()) << '\n';
    std::vector<int> v{1, 2, 3};
    for (std::vector<int>::const_reverse_iterator r = v.crbegin();
         r != v.crend(); ++r) {
        std::cout << *r << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

预期输出

```
original=10 next=30
advanced=20
distance=3
3 2 1
```

list 不支持 it + 2，可用 next，但内部仍要走两步。反向迭代器的 ++ 按反向顺序前进，即从最后一个元素走向第一个。

01 / vector：动态数组

头文件 <vector>。连续存储；普通顺序数据优先考虑。

API / 用法	作用与注意点	典型复杂度
`vector<T> c; / c(n, value)`	空容器 / 构造 n 个相同元素	O(1) / O(n)
`c = other; / assign(n, value)`	替换内容；assign 也支持迭代器区间	线性
`size() / empty() / capacity()`	元素数 / 是否为空 / 当前容量	O(1)
`max_size()`	理论容量上限，不代表机器可用内存	O(1)
`reserve(n)`	容量不足才扩容；不改变 size	至多 O(n)
`resize(n[, value])`	增删尾部元素，使 size 等于 n	与变化量相关；扩容可 O(n)
`shrink_to_fit()`	请求缩容；不保证实际释放容量	至多 O(n)
`operator[](i) / at(i)`	访问元素；at 越界抛异常	O(1)
`front() / back() / data()`	首元素 / 尾元素 / 连续数据指针	O(1)
`begin/end; cbegin/cend`	正向迭代器 / 只读迭代器	O(1)
`rbegin/rend; crbegin/crend`	反向迭代器 / 只读反向迭代器	O(1)
`push_back(value) / emplace_back(args...)`	尾插 / 就地构造；C++11 emplace_back 返回 void	摊还 O(1)
`pop_back()`	删除末尾；返回 void；不能用于空容器	O(1)
`insert(pos, value) / emplace(pos, args...)`	指定位置前插入，返回新元素迭代器	O(n)
`erase(pos) / erase(first, last)`	删除并返回后继迭代器	O(n)
`clear() / swap(other)`	清空元素但保留容量 / 交换内容	O(n) / 通常 O(1)

> 重新分配会使已有元素的迭代器、指针与引用全部失效。不扩容的插入也会使插入点及之后的位置失效。reserve 之后不能写入超出 size 的下标。vector<bool> 是特殊化实现，不能照搬普通 vector 的数据指针用法。

vector / 使用示例

独立 C++11 程序：`vector.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> a;
    a.reserve(8);                 // 只预留容量，size 仍是 0
    a.push_back(3);
    a.emplace_back(1);
    a.push_back(3);
    a.insert(a.begin() + 1, 2);    // 3 2 1 3
    std::sort(a.begin(), a.end()); // 1 2 3 3
    a.erase(std::unique(a.begin(), a.end()), a.end());
    for (int x : a) std::cout << x << ' ';
    std::cout << "\nsize=" << a.size() << '\n';
    a.resize(5, 9);               // 1 2 3 9 9
    std::cout << "last=" << a.back() << '\n';
    a.pop_back();
    a.clear();
    std::cout << "empty=" << a.empty() << '\n';
    return 0;
}
```

预期输出

```
1 2 3
size=3
last=9
empty=1
```

使用要点

> 先构造元素，再访问下标。unique 只返回逻辑末尾，erase 才缩小容器。clear 后 size 为 0，但容量仍保留。

02 / array：固定长度数组

头文件 <array>。std::array<T,N> 的长度属于类型，元素连续。

API / 用法	作用与注意点	典型复杂度
`array<T,N> c{}`	值初始化；整型元素初始化为 0	O(N)
`c = other`	逐元素赋值；要求类型及长度相同	O(N)
`size() / max_size() / empty()`	长度 N / 长度 N / N 是否为 0	O(1)
`operator[](i) / at(i)`	下标访问；at 检查越界	O(1)
`front() / back()`	首尾元素；N 必须大于 0	O(1)
`data()`	获取连续存储指针	O(1)
`begin/end; cbegin/cend`	正向迭代 / 只读正向迭代	O(1)
`rbegin/rend; crbegin/crend`	反向迭代 / 只读反向迭代	O(1)
`fill(value)`	将所有元素赋为 value	O(N)
`swap(other)`	交换各元素；不是仅交换一个指针	O(N)
`std::get<I>(c)`	编译期下标访问；I 必须小于 N	O(1)
`== != < <= > >=`（C++11 可用）	相等 / 字典序比较	至多 O(N)

> 没有 push_back、erase、resize、reserve、clear。std::array<int,0> 合法，但不能访问首尾元素。std::array<int,N> c; 在局部作用域中不会自动将所有 int 清零，应按需要写 c{}。

array / 使用示例

独立 C++11 程序：`array.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <array>
#include <algorithm>

int main() {
    std::array<int, 4> a{4, 2, 3, 1};
    std::sort(a.begin(), a.end());
    for (int x : a) std::cout << x << ' ';
    std::cout << "\nsize=" << a.size() << '\n';
    a.at(1) = 8;                  // 带边界检查的访问
    std::cout << "second=" << std::get<1>(a) << '\n';
    a.fill(7);                    // 长度不变，全改成 7
    std::cout << "first=" << a.front()
              << " last=" << a.back() << '\n';
    return 0;
}
```

预期输出

```
1 2 3 4
size=4
second=8
first=7 last=7
```

使用要点

> array 的长度由模板参数 4 决定，不能 push_back 或 resize。fill 修改的是元素值，不是容器长度。

03 / string：字符串

头文件 <string>。std::string 按 char 元素管理字符序列。

API / 用法	作用与注意点	典型复杂度
`string s; / s(n, ch); / s("abc")`	构造空串 / 重复字符 / 字符串	与构造长度相关
`size() / length() / empty()`	长度 / 长度 / 是否为空	O(1)
`capacity() / reserve(n)`	查询容量 / 预留容量	O(1) / 可线性
`resize(n[, ch]) / shrink_to_fit()`	改变长度 / 请求缩容	通常线性
`operator[](i) / at(i)`	访问字符；普通字符索引为 [0,size)	O(1)
`front() / back()`	访问首尾字符；要求非空	O(1)
`c_str() / data()`	C++11 均返回 const char*；不能通过该指针修改字符串	O(1)
`begin/end; rbegin/rend`	正向 / 反向迭代；也有 c 前缀只读版本	O(1)
`operator+= / append(...)`	追加字符串、字符或区间	通常与追加量相关；可扩容
`push_back(ch) / pop_back()`	追加 / 删除尾部字符	尾插通常摊还常数
`assign(...)`	替换整个字符串内容	通常线性
`insert(pos, text)`	在下标 pos 前插入文本	通常线性
`erase(pos, count) / erase(it)`	删除字符区间 / 单个位置	通常线性
`replace(pos, count, text)`	替换指定区间	通常线性
`substr(pos, count)`	复制子串；count 是长度	O(子串长度)
`find(text[, pos]) / rfind(...)`	正向 / 反向找子串；失败返回 npos	无统一标准复杂度保证
`find_first_of / find_last_of`	查找属于给定字符集合的字符	依实现及输入
`find_first_not_of / find_last_not_of`	查找不属于给定集合的字符	依实现及输入
`compare(...) / == / <`	比较相等或字典序	至多线性
`clear() / swap(other)`	清空 / 交换内容	通常常数 / 通常 O(1)

> string 的很多修改和搜索接口没有统一的标准复杂度承诺，表中“通常”是实现层面的常见成本。npos 应先判断再作为下标使用；UTF-8 文本不能用逐字节反转来反转汉字。

要修改已有字符，使用合法下标 s[i] 或迭代器；传出可写缓冲区时，可在非空且已正确 resize 的字符串上使用 &s[0]，并严格限制写入范围。

string / 使用示例

独立 C++11 程序：`string.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <string>

int main() {
    std::string s = "hello";
    s += " world";
    std::string::size_type pos = s.find("world");
    if (pos != std::string::npos) {
        std::cout << s.substr(pos, 5) << '\n';
        s.replace(pos, 5, "STL");
    }
    s.insert(0, "C++: ");
    std::cout << s << '\n';
    s.erase(0, 5);                // 删除前 5 个字符
    s.push_back('!');
    std::cout << s << '\n';
    std::cout << "missing="
              << (s.find("xyz") == std::string::npos) << '\n';
    return 0;
}
```

预期输出

```
world
C++: hello STL
hello STL!
missing=1
```

使用要点

> find 的结果必须先和 npos 比较。substr、replace、erase 的 count 参数是数量，不是结束下标。这里使用 ASCII 字符，避免混淆 UTF-8 字符边界。

04 / deque：双端队列

头文件 <deque>。支持随机访问，但不保证整体连续。

API / 用法	作用与注意点	典型复杂度
`deque<T> c; / c(n, value)`	构造空容器 / n 个元素	O(1) / O(n)
`assign(...) / operator=`	替换内容	线性
`size() / empty() / max_size()`	元素数 / 判空 / 理论上限	O(1)
`operator[](i) / at(i)`	随机访问；at 检查越界	O(1)
`front() / back()`	首尾元素；要求非空	O(1)
`begin/end; rbegin/rend`	双向及随机访问迭代；有只读版本	O(1)
`push_front / emplace_front`	头部插入 / 就地构造	O(1)
`push_back / emplace_back`	尾部插入 / 就地构造	O(1)
`pop_front() / pop_back()`	删除首尾；返回 void；要求非空	O(1)
`insert(pos, value) / emplace(pos, ...)`	中间插入	O(n)
`erase(pos) / erase(first,last)`	删除并返回后继	最坏 O(n)
`resize(n[, value])`	改变元素数	与增删量相关
`shrink_to_fit()`	请求减少额外存储，不保证满足	至多线性
`clear() / swap(other)`	清空 / 交换内容	O(n) / 通常 O(1)

> 没有 data、capacity、reserve。两端插入使迭代器失效，但已有元素引用仍有效；中间插入会使所有元素迭代器与引用失效。删除的失效规则因位置而异，不要跨修改长期缓存 end。

C++11 的 emplace_front / emplace_back 返回 void；位置版 emplace 返回迭代器。

deque / 使用示例

独立 C++11 程序：`deque.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> d{2, 3};
    d.push_front(1);
    d.push_back(4);               // 1 2 3 4
    std::cout << "front=" << d.front()
              << " back=" << d.back() << '\n';
    d.pop_front();
    d.pop_back();                 // 2 3
    d.insert(d.begin() + 1, 9);    // 2 9 3
    std::cout << "middle=" << d.at(1) << '\n';
    for (int x : d) std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

预期输出

```
front=1 back=4
middle=9
2 9 3
```

使用要点

> deque 同时支持两端增删和随机访问。修改后重新取得迭代器，不要继续使用插入前保存的迭代器。

05 / list：双向链表

头文件 <list>。已知位置的节点插入删除快，但寻找位置通常需要遍历。

API / 用法	作用与注意点	典型复杂度
`list<T> c; / c(n,value) / assign(...)`	构造 / 替换内容	与元素数相关
`size() / empty() / max_size()`	元素数 / 判空 / 理论上限	O(1)，C++11 起 size 常数
`front() / back()`	首尾元素	O(1)
`begin/end; rbegin/rend`	双向迭代；也有只读版本	O(1)
`push_front / push_back`	首尾插入	O(1)
`emplace_front / emplace_back`	首尾就地构造	O(1)
`pop_front / pop_back`	删除首尾，要求非空	O(1)
`insert(pos,value) / emplace(pos,...)`	在已知迭代器前插入	单元素 O(1)
`erase(pos) / erase(first,last)`	删除并返回后继	O(1) / O(k)
`resize(n[, value])`	改变长度	通常线性
`remove(value) / remove_if(pred)`	直接删除所有匹配节点	O(n)
`unique([pred])`	删除相邻重复节点，不是全局去重	O(n)
`sort([comp])`	链表成员排序；保持等价元素原顺序	O(n log n) 次比较
`reverse()`	反转节点顺序	O(n)
`merge(other[,comp])`	合并两个按相同比较器有序的链表	O(n+m)
`splice(pos,other[,it 或 first,last])`	转移节点到 pos 前	整表/单节点常数；跨表区间线性
`clear() / swap(other)`	清空 / 交换	O(n) / 通常 O(1)

> 没有下标、data、reserve。std::sort 不支持 list 迭代器，应使用成员 sort。merge/splice 要求分配器兼容；自拼接还有位置约束。插入不使已有迭代器失效，删除只使被删节点失效。

C++11 的 emplace_front / emplace_back 返回 void；位置版 emplace 返回迭代器。

list / 使用示例

独立 C++11 程序：`list.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <list>

int main() {
    std::list<int> a{3, 1, 2, 2};
    a.sort();
    a.unique();                  // 1 2 3
    std::list<int>::iterator it = a.begin();
    ++it;                        // 指向 2
    a.insert(it, 9);              // 1 9 2 3
    it = a.erase(it);             // 删除 2；it 指向 3
    std::list<int> extra{7, 8};
    a.splice(a.end(), extra);     // 转移节点，extra 变空
    a.remove(9);
    for (int x : a) std::cout << x << ' ';
    std::cout << "\nsource empty=" << extra.empty() << '\n';
    return 0;
}
```

预期输出

```
1 3 7 8
source empty=1
```

使用要点

> list 的插入不会使原有节点迭代器失效。删除后使用 erase 的返回值；排序调用成员 sort，而不是 std::sort。

06 / forward_list：单向链表

头文件 <forward_list>。操作围绕“前驱”进行。

API / 用法	作用与注意点	典型复杂度
`forward_list<T> c; / assign(...)`	构造 / 替换内容	与元素数相关
`empty() / max_size()`	判空 / 理论上限；没有 size()	O(1)
`front()`	首元素；要求非空	O(1)
`before_begin() / cbefore_begin()`	首元素之前的哨兵位置，不能解引用	O(1)
`begin/end; cbegin/cend`	只支持前向迭代，不支持 --	O(1)
`push_front / emplace_front`	头插 / 头部就地构造	O(1)
`pop_front()`	删除首元素	O(1)
`insert_after(pos,value)`	在 pos 后插入，返回新元素位置	单元素 O(1)
`emplace_after(pos,args...)`	在 pos 后就地构造	O(1)
`erase_after(pos)`	删除 pos 后的元素，返回后继	O(1)
`erase_after(first,last)`	删除开区间 (first,last) 内的节点	O(k)
`resize(n[,value])`	调整长度，通常需要遍历	线性
`remove / remove_if / unique`	按值或条件删除 / 删除相邻重复	O(n)
`sort / reverse / merge`	排序 / 反转 / 合并有序链表	O(n log n) / O(n) / O(n+m)
`splice_after(pos,other,...)`	把节点转移至 pos 后	依重载；单节点常数、整表线性
`clear() / swap(other)`	清空 / 交换	O(n) / 通常 O(1)

> 没有 size、back、push_back、反向迭代器。统计节点数可用 std::distance(begin,end)，但它是 O(n)。erase_after 的区间不删除 first 本身，这与常见 erase(first,last) 不同。

C++11 的 emplace_front 返回 void，emplace_after 返回迭代器。

forward_list / 使用示例

独立 C++11 程序：`forward_list.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <forward_list>
#include <iterator>

int main() {
    std::forward_list<int> a{2, 3};
    a.insert_after(a.before_begin(), 1); // 1 2 3
    std::forward_list<int>::iterator prev = a.begin();               // 指向 1
    a.erase_after(prev);                 // 删除 2
    a.push_front(4);                     // 4 1 3
    a.sort();                            // 1 3 4
    a.remove_if([](int x) { return x > 3; });
    for (int x : a) std::cout << x << ' ';
    std::cout << "\ncount="
              << std::distance(a.begin(), a.end()) << '\n';
    return 0;
}
```

预期输出

```
1 3
count=2
```

使用要点

> 单向链表删除需要前驱。before_begin 是首元素之前的位置，不能解引用。没有 size()；distance 统计长度需要线性遍历。

pair：两个值的组合

头文件 <utility>。pair 是保存两个值的工具类型，不是可变长容器。对于熟悉 C 的读者，可以先把它理解为一个包含 first 和 second 两个字段的结构体。

API / 类型	含义
std::pair<T1,T2> p{a,b}	保存两个值；类型可以相同，也可以不同
p.first / p.second	分别访问第一个值和第二个值；它们是成员变量，不是函数
std::make_pair(a,b)	构造 pair，由实参推导成员类型；通常按值保存
p = other	给两个成员赋值；成员类型必须支持相应赋值
p.swap(other) / std::swap(p,other)	交换两个 pair 的对应成员
std::get<0>(p) / std::get<1>(p)	按编译期位置获取成员；也可直接用 first / second
p == other / p != other	比较两个成员是否都相等 / 是否不全相等
p < other (C++11 可用)	字典序：先比较 first，first 等价时再比较 second
std::pair<const Key,Value>	map / multimap 的元素类型：key 不可直接修改，value 可修改
std::pair<Iterator,Iterator>	保存两个位置，例如 equal_range 返回的区间起点与终点
std::pair<Iterator,bool>	map / set 等唯一键容器 insert 的典型返回类型：位置 + 是否插入成功

与 C 结构体对应理解

```
struct Record {
    int first;
    /* 第二个字段可以是另一种类型 */
};
// std::pair<int, std::string> 的两个字段是 int 和 string。
```

成员访问本身是 O(1)；复制、比较、交换的成本取决于成员类型。例如字符串比较不是固定常数时间。pair 没有 push_back、begin、end。std::make_pair 会推导类型，但不要求你使用 auto。

pair / 使用示例

独立 C++11 程序：`pair.cpp`。本例显式写出类型，通过 first/second 访问成员。

```
#include <iostream>
#include <string>
#include <utility>

int main() {
    std::pair<int, std::string> person(1001, "Alice");
    std::cout << person.first << ' ' << person.second << '\n';
    person.second = "Bob";

    std::pair<int, std::string> other =
        std::make_pair(1002, std::string("Carol"));
    person.swap(other);
    std::cout << person.first << ' ' << person.second << '\n';
    std::cout << other.first << ' ' << other.second << '\n';

    std::pair<int, int> a(1, 9);
    std::pair<int, int> b(2, 0);
    std::cout << "a < b: " << (a < b) << '\n';
    std::cout << "first=" << std::get<0>(a) << '\n';
}
```

预期输出

```
1001 Alice
1002 Carol
1001 Bob
a < b: 1
first=1
```

a 的 first 为 1，b 的 first 为 2，因此 a < b 成立，不再比较 second 的 9 和 0。first、second 的名字与“键值”无关，具体含义由保存的类型和业务决定。

07 / map：有序键值表

头文件 <map>。唯一键，按比较器排序，元素类型为 pair<const Key,T>。

API / 用法	作用与注意点	典型复杂度
`map<Key,T> m; / 自定义 Compare`	构造；通常使用平衡搜索树	依构造方式
`size / empty / max_size`	元素数 / 判空 / 上限	O(1)
`begin/end; rbegin/rend`	按键顺序遍历；有只读版本	O(1)
`operator[](key)`	返回值引用；缺失则插入默认构造值	O(log n)
`at(key)`	返回值引用；缺失抛 out_of_range	O(log n)
`find(key)`	返回键的位置；失败为 end	O(log n)
`count(key)`	匹配数量；唯一键容器返回 0 或 1	O(log n)
`insert(value) / emplace(args...)`	已有键不覆盖；返回 pair<iterator,bool>	O(log n)
`insert(hint,value) / emplace_hint(...)`	提示插入；提示正确时可降低成本	通常 O(log n)
`erase(key)`	删除指定键；返回删除数	O(log n)
`erase(it) / erase(first,last)`	删除并返回后继	摊还 O(1) / O(log n+k)
`lower_bound / upper_bound / equal_range`	首个不小于 / 大于 / 等价键区间	O(log n)
`key_comp() / value_comp()`	获取键 / 元素比较器	O(1)
`clear() / swap(other)`	清空 / 交换	O(n) / 通常 O(1)

> 不要用 [] 做只读存在性检查；它可能插入新键。插入不使已有迭代器失效，删除只使被删元素失效。成员 lower_bound 比在 map 迭代器上调用通用 lower_bound 更合适。

map / 使用示例

独立 C++11 程序：`map.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <map>
#include <string>
#include <utility>

int main() {
    std::map<int, std::string> m;
    m.emplace(2, "two");
    m[1] = "one";
    std::pair<std::map<int, std::string>::iterator, bool> result = m.insert({2, "TWO"}); // 不覆盖已有键
    std::cout << "inserted=" << result.second << '\n';
    m[2] = "TWO"; // 存在则覆盖，不存在则插入
    std::map<int, std::string>::iterator it = m.find(2);
    if (it != m.end()) std::cout << it->second << '\n';
    std::map<int, std::string>::iterator bound = m.lower_bound(2);
    if (bound != m.end()) std::cout << "bound=" << bound->first << '\n';
    for (std::map<int, std::string>::const_iterator p = m.begin();
         p != m.end(); ++p)
        std::cout << p->first << ':' << p->second << '\n';
    m.erase(1);
    std::cout << "size=" << m.size() << '\n';
    return 0;
}
```

预期输出

```
inserted=0
TWO
bound=2
1:one
2:TWO
size=1
```

使用要点

insert/emplace 不覆盖已有键；m[key] = value 可插入或覆盖。[] 在键缺失时会插入；只读查询用 find 或 at。遍历按键排序。

08 / multimap：允许重复键

头文件 <map>。按键有序，多个元素可以拥有等价键。

API / 用法	作用与注意点	典型复杂度
`multimap<Key,T> m`	构造有序、多值键值表	依构造方式
`size / empty / max_size`	基本容量查询	O(1)
`begin/end; rbegin/rend`	有序遍历；支持只读版本	O(1)
`insert(value) / emplace(args...)`	允许重复键；返回 iterator，而不是 bool 对	O(log n)
`insert(hint,value) / emplace_hint(...)`	带位置提示插入	通常 O(log n)
`find(key)`	找到一个匹配元素；遍历全部匹配用 equal_range	O(log n)
`count(key)`	统计所有匹配元素	O(log n+k)
`lower_bound / upper_bound`	匹配区间的左右边界	O(log n)
`equal_range(key)`	返回包含全部等价键的半开区间	O(log n)
`erase(key)`	删除所有匹配键，返回删除数	O(log n+k)
`erase(it) / erase(first,last)`	删除一个 / 一个区间	摊还 O(1) / O(log n+k)
`key_comp / value_comp`	获取比较器	O(1)
`clear / swap`	清空 / 交换	O(n) / 通常 O(1)

> 没有 operator[]、at，因为一个键可能对应多个值。erase(key) 会删除全部同键项，只删一个时应先 find 再 erase(it)。

multimap / 使用示例

独立 C++11 程序：`multimap.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <map>
#include <string>
#include <utility>

int main() {
    std::multimap<int, std::string> m;
    m.emplace(1, "read");
    m.emplace(1, "write");
    m.emplace(2, "close");
    std::pair<std::multimap<int, std::string>::iterator,
              std::multimap<int, std::string>::iterator> range = m.equal_range(1);
    std::multimap<int, std::string>::iterator first = range.first;
    std::multimap<int, std::string>::iterator last = range.second;
    for (std::multimap<int, std::string>::iterator it = first; it != last; ++it)
        std::cout << it->second << '\n';
    std::cout << "count=" << m.count(1) << '\n';
    if (first != last) m.erase(first); // 只删一个
    std::cout << "left=" << m.count(1) << '\n';
    std::cout << "removed=" << m.erase(1) << '\n';
    return 0;
}
```

预期输出

```
read
write
count=2
left=1
removed=1
```

使用要点

> equal_range 返回全部等价键的区间。erase(it) 删除一个元素；erase(key) 删除该键的全部元素。multimap 没有 [] 和 at。

multimap / 指定 key 的所有 value

完整独立程序：multimap_values.cpp。直接复制本页代码即可编译。

```
#include <iostream>
#include <map>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::multimap<std::string, std::string> m;
    m.emplace("abc", "cba");
    m.emplace("abc", "bca");
    m.emplace("def", "fed");
    std::pair<
        std::multimap<std::string, std::string>::const_iterator,
        std::multimap<std::string, std::string>::const_iterator
    > range = m.equal_range("abc");
    std::vector<std::string> values;
    for (std::multimap<std::string, std::string>::const_iterator
             it = range.first; it != range.second; ++it) {
        values.push_back(it->second);
    }
    std::cout << "abc:";
    for (std::size_t i = 0; i < values.size(); ++i)
        std::cout << ' ' << values[i];
    std::cout << '\n';
    return 0;
}
```

预期输出

```
abc: cba bca
```

range.first / range.second 是分组起点和终点；it->first / it->second 是当前元素的 key 和 value。没有匹配键时，两个边界相等，得到空 vector。

multimap / 遍历所有 key 并分组

完整独立程序：multimap_group.cpp。不依赖其他示例或公共类型定义。

```
#include <iostream>
#include <map>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::multimap<std::string, std::string> m;
    m.emplace("abc", "cba");
    m.emplace("abc", "bca");
    m.emplace("def", "fed");
    std::multimap<std::string, std::string>::const_iterator it
        = m.begin();
    while (it != m.end()) {
        std::string key = it->first;
        std::pair<
            std::multimap<std::string, std::string>::const_iterator,
            std::multimap<std::string, std::string>::const_iterator
        > range = m.equal_range(key);
        std::vector<std::string> values;
        for (std::multimap<std::string, std::string>::const_iterator
                 p = range.first; p != range.second; ++p)
            values.push_back(p->second);
        std::cout << key << ':';
        for (std::size_t i = 0; i < values.size(); ++i)
            std::cout << ' ' << values[i];
        std::cout << '\n';
        it = range.second;
    }
    return 0;
}
```

预期输出

```
abc: cba bca
def: fed
```

it = range.second 使外层循环跳到下一个不同的 key，避免重复处理同一组。每组 values 都保存原 value 的副本。

09 / set：有序集合

头文件 <set>。键唯一。集合中的键不能通过迭代器直接修改。

API / 用法	作用与注意点	典型复杂度
`set<Key> s`	构造；可以指定比较器	依构造方式
`size / empty / max_size`	元素数 / 判空 / 上限	O(1)
`begin/end; rbegin/rend`	按键顺序遍历；有只读版本	O(1)
`insert(value) / emplace(args...)`	返回 pair<iterator,bool>	O(log n)
`insert(hint,value) / emplace_hint(...)`	带提示插入	通常 O(log n)
`find(key)`	匹配位置或 end	O(log n)
`count(key)`	匹配数量，只能为 0 或 1	O(log n)
`lower_bound / upper_bound / equal_range`	有序边界查询 / 等价键区间	O(log n)
`erase(key)`	按键删除	O(log n)
`erase(it) / erase(first,last)`	删除一个 / 一段，返回后继	摊还 O(1) / O(log n+k)
`key_comp / value_comp`	获取比较器	O(1)
`clear / swap`	清空 / 交换	O(n) / 通常 O(1)

> 没有下标、at、push_back。键等价由比较器决定，不一定等同于 operator==。修改键应先保存所需内容，删除原元素，再插入修改后的值。

set / 使用示例

独立 C++11 程序：`set.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <set>
#include <utility>

int main() {
    std::set<int> s{3, 1, 3};     // 去掉重复的 3
    std::pair<std::set<int>::iterator, bool> result = s.insert(2);
    std::cout << "inserted=" << result.second << '\n';
    std::cout << "has3=" << (s.find(3) != s.end()) << '\n';
    std::set<int>::iterator bound = s.lower_bound(2);
    if (bound != s.end()) std::cout << "bound=" << *bound << '\n';
    s.erase(1);
    for (int x : s) std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

预期输出

```
inserted=1
has3=1
bound=2
2 3
```

使用要点

> set 既去重又排序。不能通过 *it 修改键；需要修改时，先保存值，删除后再插入修改后的值。

10 / multiset：有序集合

头文件 <set>。允许重复键。集合中的键不能通过迭代器直接修改。

API / 用法	作用与注意点	典型复杂度
`multiset<Key> s`	构造；可以指定比较器	依构造方式
`size / empty / max_size`	元素数 / 判空 / 上限	O(1)
`begin/end; rbegin/rend`	按键顺序遍历；有只读版本	O(1)
`insert(value) / emplace(args...)`	返回 iterator	O(log n)
`insert(hint,value) / emplace_hint(...)`	带提示插入	通常 O(log n)
`find(key)`	匹配位置或 end	O(log n)
`count(key)`	匹配数量	O(log n+k)
`lower_bound / upper_bound / equal_range`	有序边界查询 / 等价键区间	O(log n)
`erase(key)`	按键删除全部重复项	O(log n+k)
`erase(it) / erase(first,last)`	删除一个 / 一段，返回后继	摊还 O(1) / O(log n+k)
`key_comp / value_comp`	获取比较器	O(1)
`clear / swap`	清空 / 交换	O(n) / 通常 O(1)

> 没有下标、at、push_back。键等价由比较器决定，不一定等同于 operator==。修改键应先保存所需内容，删除原元素，再插入修改后的值。

multiset / 使用示例

独立 C++11 程序：`multiset.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <set>

int main() {
    std::multiset<int> s{2, 2, 2, 3};
    std::cout << "count=" << s.count(2) << '\n';
    std::multiset<int>::iterator it = s.find(2);
    if (it != s.end()) s.erase(it); // 只删一个 2
    std::cout << "after one=" << s.count(2) << '\n';
    std::cout << "removed=" << s.erase(2) << '\n';
    for (int x : s) std::cout << x << ' ';
    std::cout << '\n';
    return 0;
}
```

预期输出

```
count=3
after one=2
removed=2
3
```

使用要点

> 重复值是独立元素。最容易写错的是把 erase(value) 当作只删除一次；它实际上删除全部等价键。

11 / unordered_map

头文件 <unordered_map>。键唯一。

API / 用法	作用与注意点	典型复杂度
unordered_map<Key,T>	哈希容器，无排序遍历保证	依构造方式
size / empty / max_size	元素数 / 判空 / 理论上限	O(1)
begin/end; cbegin/cend	前向迭代；没有 rbegin/rend	O(1)
find(key)	匹配位置或 end	平均 O(1)，最坏 O(n)
count(key)	匹配数量，0 或 1	平均 O(1)，最坏 O(n)
insert / emplace	返回 pair<iterator,bool>	平均 O(1)，最坏 O(n)
erase(key)	删除匹配键	平均 O(1)，最坏 O(n)
erase(it) / erase(first,last)	删除一个 / 区间，返回后继	平均与删除量相关，最坏 O(n)
equal_range(key)	全部等价键组成的区间	平均 O(k)，最坏 O(n)
operator[] / at	[] 缺失会插入；at 缺失抛异常	平均 O(1)，最坏 O(n)
reserve(n)	为预计 n 个元素规划桶数	平均线性，最坏二次
rehash(buckets)	请求至少相应桶数，并满足负载约束	平均线性，最坏二次
load_factor / max_load_factor	当前平均每桶元素数 / 读写负载上限	O(1)
bucket_count / bucket(key) / bucket_size(i)	桶数 / 键所在桶 / 某桶元素数	常数 / 哈希成本 / 桶长
hash_function / key_eq	获取哈希函数和等价判断器	O(1)
clear / swap	清空 / 交换	O(n) / 通常 O(1)

> rehash 使迭代器失效，但不会使已有元素指针和引用失效。等价键必须产生相同 hash；哈希相同仍需比较键。没有 lower_bound/upper_bound。

unordered_map / 使用示例

独立 C++11 程序：`unordered_map.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <unordered_map>
#include <string>

int main() {
    std::unordered_map<std::string, int> freq;
    freq.reserve(32);
    ++freq["apple"];
    ++freq["banana"];
    ++freq["apple"];
    std::cout << "apple=" << freq.at("apple") << '\n';
    std::unordered_map<std::string, int>::iterator it = freq.find("pear");
    std::cout << "has pear=" << (it != freq.end()) << '\n';
    freq.emplace("apple", 99); // 已有键不覆盖
    freq["banana"] = 5;
    std::cout << "banana=" << freq.at("banana") << '\n';
    freq.erase("banana");
    std::cout << "size=" << freq.size() << '\n';
    return 0;
}
```

预期输出

```
apple=2
has pear=0
banana=5
size=1
```

使用要点

> reserve 按预计元素数设置容量规划。平均查找常数复杂度不等于最坏常数复杂度。示例不依赖哈希桶数量或遍历顺序。

12 / unordered_multimap

头文件 <unordered_map>。允许重复键。

API / 用法	作用与注意点	典型复杂度
`unordered_multimap<Key,T>`	哈希容器，无排序遍历保证	依构造方式
`size / empty / max_size`	元素数 / 判空 / 理论上限	O(1)
`begin/end; cbegin/cend`	前向迭代；没有 rbegin/rend	O(1)
`find(key)`	匹配位置或 end	平均 O(1)，最坏 O(n)
`count(key)`	匹配数量	平均 O(k)，最坏 O(n)
`insert / emplace`	返回 iterator	平均 O(1)，最坏 O(n)
`erase(key)`	删除全部匹配项	平均 O(k)，最坏 O(n)
`erase(it) / erase(first,last)`	删除一个 / 区间，返回后继	平均与删除量相关，最坏 O(n)
`equal_range(key)`	全部等价键组成的区间	平均 O(k)，最坏 O(n)
`reserve(n)`	为预计 n 个元素规划桶数	平均线性，最坏二次
`rehash(buckets)`	请求至少相应桶数，并满足负载约束	平均线性，最坏二次
`load_factor / max_load_factor`	当前平均每桶元素数 / 读写负载上限	O(1)
`bucket_count / bucket(key) / bucket_size(i)`	桶数 / 键所在桶 / 某桶元素数	常数 / 哈希成本 / 桶长
`hash_function / key_eq`	获取哈希函数和等价判断器	O(1)
`clear / swap`	清空 / 交换	O(n) / 通常 O(1)

> rehash 使迭代器失效，但不会使已有元素指针和引用失效。等价键必须产生相同 hash；哈希相同仍需比较键。没有 lower_bound/upper_bound。没有 [] 或 at。

unordered_multimap / 使用示例

独立 C++11 程序：`unordered_multimap.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <unordered_map>
#include <string>
#include <vector>
#include <algorithm>
#include <utility>

int main() {
    std::unordered_multimap<int, std::string> m;
    m.emplace(7, "write");
    m.emplace(7, "read");
    m.emplace(9, "close");
    std::pair<std::unordered_multimap<int, std::string>::iterator,
              std::unordered_multimap<int, std::string>::iterator> range
        = m.equal_range(7);
    std::vector<std::string> values;
    for (std::unordered_multimap<int, std::string>::iterator it
             = range.first; it != range.second; ++it)
        values.push_back(it->second);
    // 为得到稳定输出，对查到的值排序；不是哈希容器自带排序
    std::sort(values.begin(), values.end());
    for (const std::string& value : values) std::cout << value << '\n';
    std::cout << "count=" << m.count(7) << '\n';
    std::cout << "removed=" << m.erase(7) << '\n';
    return 0;
}
```

预期输出

```
read
write
count=2
removed=2
```

使用要点

一个键可以对应多个值，使用 equal_range 获取全部匹配项；没有 [] 或 at。排序只为使演示输出可复现，不是查询本身的必要步骤。

13 / unordered_set

头文件 <unordered_set>。键唯一。

API / 用法	作用与注意点	典型复杂度
`unordered_set<Key>`	哈希容器，无排序遍历保证	依构造方式
`size / empty / max_size`	元素数 / 判空 / 理论上限	O(1)
`begin/end; cbegin/cend`	前向迭代；没有 rbegin/rend	O(1)
`find(key)`	匹配位置或 end	平均 O(1)，最坏 O(n)
`count(key)`	匹配数量，0 或 1	平均 O(1)，最坏 O(n)
`insert / emplace`	返回 pair<iterator,bool>	平均 O(1)，最坏 O(n)
`erase(key)`	删除匹配键	平均 O(1)，最坏 O(n)
`erase(it) / erase(first,last)`	删除一个 / 区间，返回后继	平均与删除量相关，最坏 O(n)
`equal_range(key)`	全部等价键组成的区间	平均 O(k)，最坏 O(n)
`reserve(n)`	为预计 n 个元素规划桶数	平均线性，最坏二次
`rehash(buckets)`	请求至少相应桶数，并满足负载约束	平均线性，最坏二次
`load_factor / max_load_factor`	当前平均每桶元素数 / 读写负载上限	O(1)
`bucket_count / bucket(key) / bucket_size(i)`	桶数 / 键所在桶 / 某桶元素数	常数 / 哈希成本 / 桶长
`hash_function / key_eq`	获取哈希函数和等价判断器	O(1)
`clear / swap`	清空 / 交换	O(n) / 通常 O(1)

> rehash 使迭代器失效，但不会使已有元素指针和引用失效。等价键必须产生相同 hash；哈希相同仍需比较键。没有 lower_bound/upper_bound。没有 [] 或 at。

unordered_set / 使用示例

独立 C++11 程序：`unordered_set.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <unordered_set>
#include <utility>

int main() {
    std::unordered_set<int> s;
    s.reserve(16);
    std::pair<std::unordered_set<int>::iterator, bool> first = s.insert(3);
    std::pair<std::unordered_set<int>::iterator, bool> again = s.insert(3);
    s.insert(1);
    std::cout << "first=" << first.second
              << " duplicate=" << again.second << '\n';
    std::cout << "has3=" << (s.find(3) != s.end()) << '\n';
    s.erase(3);
    std::cout << "count3=" << s.count(3)
              << " size=" << s.size() << '\n';
    return 0;
}
```

预期输出

```
first=1 duplicate=0
has3=1
count3=0 size=1
```

使用要点

> 适合去重与成员存在性检查，不提供排序和范围边界查找。插入可能触发 rehash，因此不要跨插入操作依赖旧迭代器。

14 / unordered_multiset

头文件 <unordered_set>。允许重复键。

API / 用法	作用与注意点	典型复杂度
`unordered_multiset<Key>`	哈希容器，无排序遍历保证	依构造方式
`size / empty / max_size`	元素数 / 判空 / 理论上限	O(1)
`begin/end; cbegin/cend`	前向迭代；没有 rbegin/rend	O(1)
`find(key)`	匹配位置或 end	平均 O(1)，最坏 O(n)
`count(key)`	匹配数量	平均 O(k)，最坏 O(n)
`insert / emplace`	返回 iterator	平均 O(1)，最坏 O(n)
`erase(key)`	删除全部匹配项	平均 O(k)，最坏 O(n)
`erase(it) / erase(first,last)`	删除一个 / 区间，返回后继	平均与删除量相关，最坏 O(n)
`equal_range(key)`	全部等价键组成的区间	平均 O(k)，最坏 O(n)
`reserve(n)`	为预计 n 个元素规划桶数	平均线性，最坏二次
`rehash(buckets)`	请求至少相应桶数，并满足负载约束	平均线性，最坏二次
`load_factor / max_load_factor`	当前平均每桶元素数 / 读写负载上限	O(1)
`bucket_count / bucket(key) / bucket_size(i)`	桶数 / 键所在桶 / 某桶元素数	常数 / 哈希成本 / 桶长
`hash_function / key_eq`	获取哈希函数和等价判断器	O(1)
`clear / swap`	清空 / 交换	O(n) / 通常 O(1)

> rehash 使迭代器失效，但不会使已有元素指针和引用失效。等价键必须产生相同 hash；哈希相同仍需比较键。没有 lower_bound/upper_bound。没有 [] 或 at。

unordered_multiset / 使用示例

独立 C++11 程序：`unordered_multiset.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <unordered_set>
#include <iterator>
#include <utility>

int main() {
    std::unordered_multiset<int> s{2, 2, 2, 3};
    std::cout << "count=" << s.count(2) << '\n';
    std::pair<std::unordered_multiset<int>::iterator,
              std::unordered_multiset<int>::iterator> range
        = s.equal_range(2);
    std::cout << "range=" << std::distance(range.first, range.second) << '\n';
    std::unordered_multiset<int>::iterator it = s.find(2);
    if (it != s.end()) s.erase(it);
    std::cout << "after one=" << s.count(2) << '\n';
    std::cout << "removed=" << s.erase(2) << '\n';
    std::cout << "size=" << s.size() << '\n';
    return 0;
}
```

预期输出

```
count=3
range=3
after one=2
removed=2
size=1
```

使用要点

> 允许重复元素，但仍不保证全局排序。erase(it) 删除一个，erase(key) 删除全部。区间迭代器应在结构变化前使用。

15 / stack：容器适配器

头文件 <stack>。接口受适配器约束，底层容器接口并不全部暴露。

API / 用法	作用与注意点	典型复杂度
`stack<T> s`	默认底层 deque<T>	依构造方式
`empty() / size()`	判空 / 元素数	O(1)
`top()`	返回栈顶引用；要求非空	O(1)
`push(value) / emplace(args...)`	压栈 / 就地构造	默认底层 O(1)
`pop()`	出栈；返回 void；要求非空	O(1)
`swap(other)`	交换两个栈	通常 O(1)

> 后进先出。没有 begin/end、下标、clear。可反复 pop 清空，或与空栈交换。

C++11 的 emplace 返回 void，不应将其结果赋给变量。

stack / 使用示例

独立 C++11 程序：`stack.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> s;
    s.push(10);
    s.emplace(20);
    s.push(30);
    std::cout << "size=" << s.size() << '\n';
    while (!s.empty()) {
        int value = s.top(); // 先取值，再删除
        s.pop();             // pop 不返回被删元素
        std::cout << value << ' ';
    }
    std::cout << "\nempty=" << s.empty() << '\n';
    return 0;
}
```

预期输出

```
size=3
30 20 10
empty=1
```

使用要点

> 后进先出，适合括号匹配、撤销历史和显式模拟递归。没有迭代器；如需保留原栈，可以复制一份再弹出检查。

16 / queue：容器适配器

头文件 <queue>。接口受适配器约束，底层容器接口并不全部暴露。

API / 用法	作用与注意点	典型复杂度
`queue<T> q`	默认底层 deque<T>	依构造方式
`empty() / size()`	判空 / 元素数	O(1)
`front() / back()`	队首 / 队尾引用；要求非空	O(1)
`push(value) / emplace(args...)`	在队尾插入	默认底层 O(1)
`pop()`	删除队首；返回 void	O(1)
`swap(other)`	交换队列	通常 O(1)

> 先进先出。没有 top、begin/end、下标、clear。BFS 应在入队时标记访问状态，避免重复入队。

C++11 的 emplace 返回 void，不应将其结果赋给变量。

queue / 使用示例

独立 C++11 程序：`queue.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <queue>

int main() {
    std::queue<int> q;
    q.push(10);
    q.emplace(20);
    q.push(30);
    std::cout << "front=" << q.front()
              << " back=" << q.back() << '\n';
    while (!q.empty()) {
        int value = q.front();
        q.pop();
        std::cout << value << ' ';
    }
    std::cout << "\nempty=" << q.empty() << '\n';
    return 0;
}
```

预期输出

```
front=10 back=30
10 20 30
empty=1
```

使用要点

> 先进先出，适合 BFS 和顺序待处理任务。front 读队首，back 读队尾，pop 删除队首；这几个操作都不能用于空队列。

17 / priority_queue：容器适配器

头文件 <queue>。接口受适配器约束，底层容器接口并不全部暴露。

API / 用法	作用与注意点	典型复杂度
`priority_queue<T> q`	默认 vector + less：最大元素在顶	空构造 O(1)
`priority_queue(first,last)`	用区间构造堆	通常 O(n)
`priority_queue<T,vector<T>,greater<T>>`	小根堆；需要 <functional>	依构造方式
`empty() / size()`	判空 / 元素数	O(1)
`top()`	返回堆顶 const 引用；要求非空	O(1)
`push(value) / emplace(args...)`	插入并维护堆	O(log n) 堆调整
`pop()`	删除堆顶；返回 void	O(log n)
`swap(other)`	交换优先队列	通常 O(1)

> push 可能触发底层 vector 扩容，单次可出现线性移动。没有迭代器、erase(key)、任意元素更新接口。默认大根堆；greater 为小根堆。比较器不能使用 <=。

priority_queue / 小根堆基础示例

独立 C++11 程序：`priority_queue.cpp`。先读上页 API 表，再对照下面的操作顺序。

```
#include <iostream>
#include <queue>
#include <vector>
#include <functional>

int main() {
    // greater<int>：数值越小，越先出队。
    std::priority_queue<int, std::vector<int>, std::greater<int>> heap;
    heap.push(5);
    heap.push(2);
    heap.emplace(3);
    heap.push(2);                   // 允许重复值
    std::cout << "min=" << heap.top()
              << " size=" << heap.size() << '\n';
    heap.pop();                     // 删除一个最小值，不返回元素
    heap.push(1);
    while (!heap.empty()) {
        int value = heap.top();     // 先复制，再 pop；不要保留引用
        heap.pop();
        std::cout << value << ' ';
    }
    std::cout << "\nempty=" << heap.empty() << '\n';
    return 0;
}
```

预期输出

```
min=2 size=4
1 2 3 5
empty=1
```

使用要点

> 小根堆的 top() 是当前最小值；默认 priority_queue 则是大根堆。top/pop 前必须判空。没有迭代器和任意元素修改接口；算法中的优先级更新可采用“重新入堆 + 跳过旧状态”，见后面的 Dijkstra 模板。

小根堆 / 可复制的通用模板

独立 C++11 程序：`min_heap_template.cpp`。复制 make_min_heap 函数；修改 vector 的元素类型即可复用。

```
#include <functional>
#include <iostream>
#include <queue>
#include <vector>

// T 需要支持 > 比较；整数、pair 等标准类型可以直接使用。
template <typename T>
std::priority_queue<T, std::vector<T>, std::greater<T>>
make_min_heap(const std::vector<T>& values) {
    return std::priority_queue<T, std::vector<T>, std::greater<T>>(
        values.begin(), values.end());
}

int main() {
    std::vector<long long> values{7, -4, 7, 2};
    std::priority_queue<long long, std::vector<long long>,
                        std::greater<long long>> heap
        = make_min_heap(values);
    heap.push(0);
    while (!heap.empty()) {
        long long value = heap.top();
        heap.pop();
        std::cout << value << ' ';  // 在此处理当前最小元素
    }
    std::cout << '\n';
}
```

预期输出

```
-4 0 2 7 7
```

区间建堆 O(n)；top/empty/size 为 O(1)，push/pop 为 O(log n) 堆调整。逐次弹出全部 n 个元素总计 O(n log n)。本例保存副本，额外空间 O(n)，原数组不变；push 的 vector 扩容另有摊销成本。

小根堆 / pair 状态与排序顺序

独立 C++11 程序：`min_heap_pair.cpp`。适合“代价 + 编号”的算法状态，如最短路、合并有序序列。

```
#include <functional>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

int main() {
    // pair.first = 距离/代价，pair.second = 节点编号。
    std::priority_queue<
        std::pair<long long, int>,
        std::vector<std::pair<long long, int>>,
        std::greater<std::pair<long long, int>>> heap;
    heap.emplace(8, 2);
    heap.emplace(3, 5);
    heap.emplace(3, 1);
    while (!heap.empty()) {
        std::pair<long long, int> state = heap.top();
        heap.pop();
        long long cost = state.first;
        int node = state.second;
        std::cout << cost << ' ' << node << '\n';
    }
}
```

预期输出

```
3 1
3 5
8 2
```

pair 按字典序比较：先比较 first，相同再比较 second。因此把优先级放在 first。若写成 pair<编号,距离>，就会先按编号出队。top 返回 const 引用，先复制状态再 pop。

小根堆 / 结构体与自定义比较器

独立 C++11 程序：`min_heap_custom.cpp`。三个及以上字段、特殊并列规则，可用结构体和比较器。

```
#include <iostream>
#include <queue>
#include <vector>

struct State {
    long long cost;
    int id;
};

struct Compare {
    bool operator()(const State& a, const State& b) const {
        if (a.cost != b.cost) return a.cost > b.cost;
        return a.id > b.id;  // 代价相等，编号小的先出队
    }
};

int main() {
    std::priority_queue<State, std::vector<State>, Compare> heap;
    heap.push({8, 2});
    heap.push({3, 5});
    heap.push({3, 1});
    while (!heap.empty()) {
        State state = heap.top();
        heap.pop();
        std::cout << state.cost << ' ' << state.id << '\n';
    }
}
```

预期输出

```
3 1
3 5
8 2
```

Compare(a,b) 为 true 表示 a 应排在 b 后面；所以小根堆写 a.cost > b.cost。比较器必须满足严格弱序，不能写 >= 或 <=。等价元素不保证 FIFO；需要按到达顺序处理时，将递增序号作为第二比较字段。不要让比较结果依赖会被外部修改的 dist 数组。

小根堆 / Top K 与第 k 大模板

独立 C++11 程序：`min_heap_topk.cpp`。维护至多 k 个候选值；始终保留已扫描元素中最大的 k 个。

```
#include <functional>
#include <iostream>
#include <queue>
#include <stdexcept>
#include <vector>

// 按元素个数排名，重复值分别计数；k 从 1 开始。
int kth_largest(const std::vector<int>& values, int k) {
    if (k <= 0 || static_cast<std::size_t>(k) > values.size())
        throw std::out_of_range("k must be in [1, n]");
    std::priority_queue<int, std::vector<int>, std::greater<int>> heap;
    for (int value : values) {
        if (heap.size() < static_cast<std::size_t>(k)) {
            heap.push(value);
        } else if (value > heap.top()) {
            heap.pop();       // 淘汰当前保留元素中的最小值
            heap.push(value);
        }
    }
    return heap.top();        // 最大的 k 个数中，最小的就是第 k 大
}

int main() {
    std::vector<int> values{3, 2, 1, 5, 6, 4};
    std::cout << kth_largest(values, 2) << '\n';
    std::vector<int> repeated{5, 5, 4};
    std::cout << kth_largest(repeated, 2) << '\n';
}
```

预期输出

```
5
5
```

第 k 大用小根堆：堆顶是当前候选中最容易被淘汰的最小值。第 k 小则用大根堆。时间 O(n log(k+1))，额外空间 O(k)；k=1 时为 O(n)。此处重复值分别计数，不是“第 k 个不同值”。

小根堆 / Dijkstra 函数模板

独立 C++11 程序：`min_heap_dijkstra.cpp`。非负边权最短路；本页为头文件和函数，下页为同一文件的 main。

```
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

struct Edge { int to; long long weight; };
const long long INF = std::numeric_limits<long long>::max() / 4;

// 顶点编号 0..n-1，起点/边端点有效；边权非负，有限最短路 < INF。
std::vector<long long> dijkstra(
    const std::vector<std::vector<Edge>>& graph, int source) {
    std::vector<long long> dist(graph.size(), INF);
    std::priority_queue<std::pair<long long, int>,
        std::vector<std::pair<long long, int>>,
        std::greater<std::pair<long long, int>>> heap;
    dist[source] = 0;
    heap.emplace(0, source);
    while (!heap.empty()) {
        std::pair<long long, int> state = heap.top();
        heap.pop();
        long long du = state.first;
        int u = state.second;
        if (du != dist[u]) continue;  // 跳过已过期的旧距离
        for (const Edge& edge : graph[u]) {
            int v = edge.to;
            long long w = edge.weight;
            if (w >= INF - du) continue;  // 避免加法溢出/越过哨兵
            long long candidate = du + w;
            if (candidate < dist[v]) {
                dist[v] = candidate;
                heap.emplace(candidate, v);  // 新状态重新入堆
            }
        }
    }
    return dist;
}
```

优先队列不提供 decrease-key。距离变小时，把新距离和节点重新入堆；弹出时用 du != dist[u] 丢弃旧状态。不要在第一次入堆时就标记“已确定最短距离”。

小根堆 / Dijkstra 运行与更新说明

接在上页函数后即可运行；完整代码已经保存在 `min_heap_dijkstra.cpp`。

```
int main() {
    std::vector<std::vector<Edge>> graph(5);
    graph[0].push_back({1, 4});  // 有向边 0 -> 1，权重 4
    graph[0].push_back({2, 1});
    graph[2].push_back({1, 2});
    graph[1].push_back({3, 1});
    graph[2].push_back({3, 5});
    std::vector<long long> dist = dijkstra(graph, 0);
    for (long long distance : dist) {
        if (distance == INF) std::cout << "INF ";
        else std::cout << distance << ' ';
    }
    std::cout << '\n';
}
```

预期输出

```
0 3 1 4 INF
```

为什么需要跳过旧状态?

节点 1 先通过 0→1 得到距离 4，入堆 (4,1)；随后通过 0→2→1 得到距离 3，再入堆 (3,1)。旧的 (4,1) 仍留在堆里，弹出时发现 4 != dist[1]，直接跳过。节点 4 不可达，保留 INF。

套用到算法题

输入顶点数、边数和起点，按输入填充 graph；无向边需要在两个方向各插入一次。有负权边时不能套用 Dijkstra。INF 是不可达哨兵，题目的有限距离必须小于它。

> 本实现允许旧状态暂留堆中：最坏时间 O(V + E log(E+1))，图、距离和堆的总空间 O(V+E)。堆可能有 O(E) 个条目，不应按只有 V 个元素估算。

18 / 常用算法：配合容器使用

头文件 <algorithm>、<numeric>、<iterator>。区间通常为 [first,last)。

API / 用法	作用与注意点	典型复杂度
`sort(first,last[,comp])`	随机访问迭代器；list 要用成员 sort	O(n log n) 次比较
`stable_sort(...)`	保留等价元素原顺序	内存足够 O(n log n)；否则 O(n log² n)
`find / find_if / count / count_if`	查找 / 条件查找 / 计数	O(n)
`lower_bound / upper_bound`	有序区间边界；与比较器一致	O(log n) 比较；移动可能 O(n)
`binary_search / equal_range`	是否存在 / 等价区间	O(log n) 比较
`unique(first,last)`	移除相邻重复，返回逻辑新末尾	O(n)
`remove / remove_if`	移动保留元素，不直接缩小容器	O(n)
`reverse / rotate / fill`	反转 / 旋转 / 填充	O(n)
`copy / transform`	复制 / 转换；确保输出空间足够	O(n)
`min_element / max_element`	返回位置；空区间返回 end	O(n)
`all_of / any_of / none_of`	谓词判断，可提前结束	至多 O(n)
`nth_element`	选择名次，不保证整体有序	常用非并行重载平均 O(n)
`accumulate(...,0LL)`	用 long long 累加；初值决定累加类型	O(n)
`iota(first,last,value)`	依次填入递增值	O(n)
`next_permutation`	下一个字典序排列	O(n)
`back_inserter(c)`	产生追加式输出迭代器；需 push_back	每次赋值调用 push_back

> unique/remove 之后常需 c.erase(new_end,c.end())。C++11 按条件删除可使用 erase-remove，或正确维护迭代器的 erase 循环。标准容器自身不会提供无同步的任意并发读写保证。

编译方式与参考资料

配套目录：STL容器独立示例_C++11。每个 .cpp 都有独立 main()，请分别编译运行。

GCC / MinGW：严格选择 C++11

```
g++ -std=c++11 -O2 -Wall -Wextra -pedantic-errors vector.cpp -o vector_demo
./vector_demo
```

Windows PowerShell 下运行：

```
.\vector_demo.exe
```

Clang

```
clang++ -std=c++11 -O2 -Wall -Wextra -pedantic-errors vector.cpp -o vector_demo
```

Visual Studio / MSVC

MSVC 没有 /std:c++11 开关；/std:c++14 也不能用于严格限制 C++11。本手册以 GCC 的 -std=c++11 编译验证为准。MSVC 用户可使用兼容模式编译，但应另外用 GCC 或 Clang 检查标准边界。

参考资料

- WG21 N3337（2012-01-16 公开工作草案，供标准条文查阅）
- GCC：语言标准与 -std 选项
- Microsoft：/std 支持的语言版本

> 本文件是按 C++11 编写的 STL 使用手册，不是 ISO 标准全文。API 表、代码和算法模板均以 C++11 可用接口为范围。